



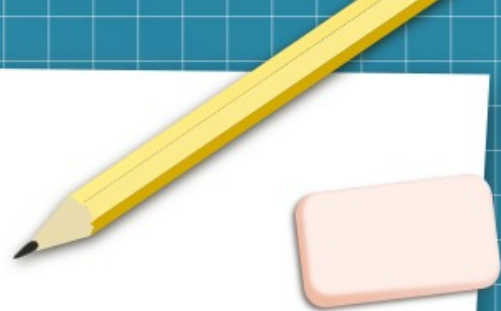
Lập trình di động Flutter

Bài 4

Khoa CNTT - Đại học Phenikaa

Nội dung trình bày

- State
- Stateful Widget
- Ví dụ
- Bài tập



State (Trạng thái)

In General

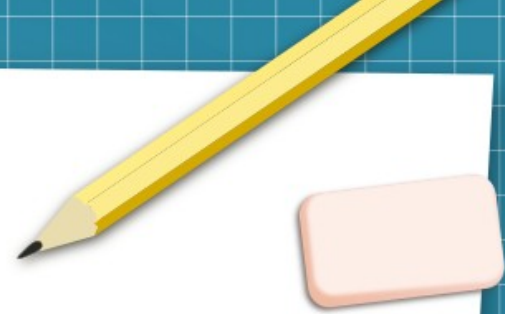
State is Data/ Information used by your App

App State

Authenticated Users
Loaded Jobs
...

Widget State

Current User Input
Is a Loading Spinner being shown?

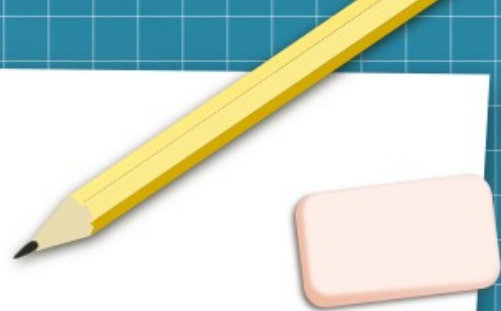


State (Trạng thái)



- Trước khi tạo một Widget, ta cần phải quyết định xem trạng thái có thay đổi qua thời gian hay không. Nếu trạng thái thay đổi, tức là có một điều gì đó đã xảy ra, chẳng hạn như:
 - Người dùng tác động vào một thành phần nào đó và làm cho một số thứ trong giao diện người dùng phải thay đổi.
 - Điện thoại xoay và giao diện người dùng phải được vẽ lại.
 - Có một sự kiện nào đó xảy ra và một Widget phụ thuộc vào sự kiện đó được thông báo (và do đó việc vẽ lại sẽ xảy ra để phản ánh những thay đổi mang lại).

State (Trạng thái)



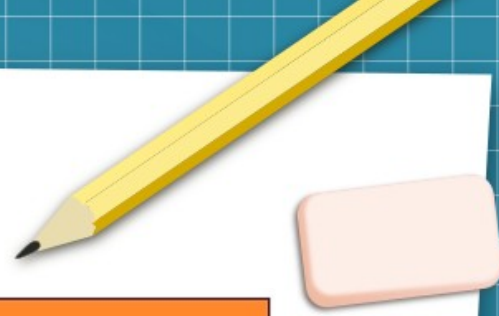
- **Stateless widget**

- Sử dụng Stateless widget khi ta cần tạo một phần giao diện người dùng không thay đổi theo thời gian.

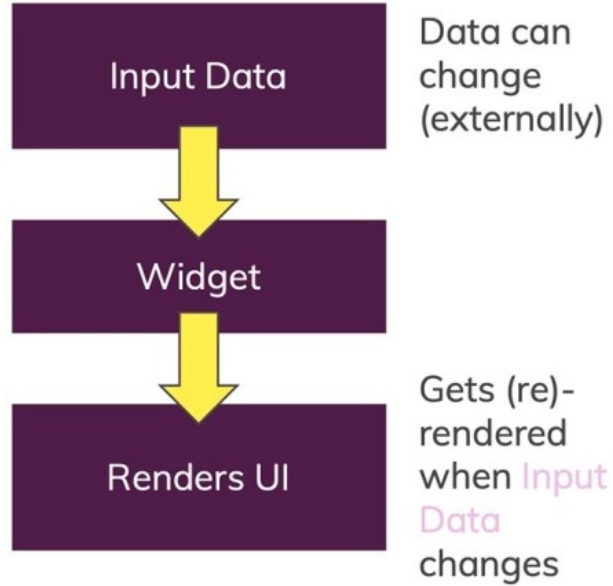
- **Stateful widget**

- Sử dụng Stateful widget khi ta cần tạo một phần giao diện người dùng sẽ thay đổi theo thời gian.
 - Giao diện người dùng sẽ thay đổi linh hoạt do các sự kiện bên ngoài, chẳng hạn như phản hồi nhận được của HTTP request hoặc nhấn nút.

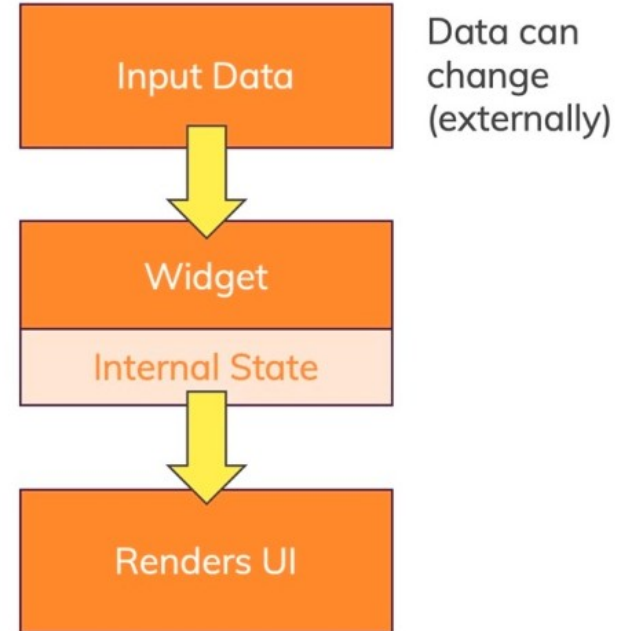
Stateless và Stateful



Stateless



Stateful



Cấu trúc chung khi xây dựng Stateful Widget



```
import 'package:flutter/material.dart';
```

```
class MyApp extends StatefulWidget {  
  @override  
  MyAppState createState() {  
    return MyAppState();  
  }  
}
```

```
class MyAppState extends State<MyApp> {  
  Widget build(BuildContext build) {  
    return MaterialApp(  
      home: Scaffold(  
        body: <Widget Tree>  
      ),  
    );  
  }  
}
```

```
main() => runApp(MyApp());
```

Stateful Widget

- Ví dụ:

```
import 'package:flutter/material.dart';

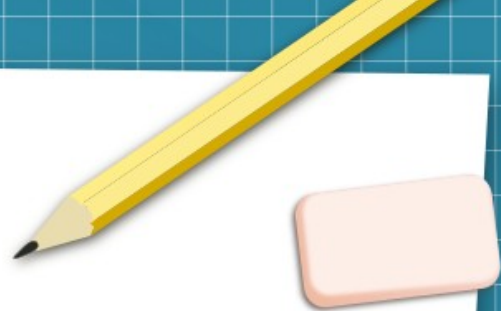
class Counter extends StatefulWidget {
  @override
  _CounterState createState() => _CounterState();
}
```

```
class _CounterState extends State<Counter> {
  int _counter = 0;

  Widget build(BuildContext build) {
    return MaterialApp(
      home: Scaffold(
        body: Center (
          child: Column (
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              Text("$ _counter",
                style: TextStyle(
                  fontSize: 46,
                ), ), // TextStyle, Text
              IconButton(
                icon: const Icon(Icons.add),
                onPressed: () {
                  setState(() => _counter++);
                }
              ), // IconButton
            ],
          ), ), ), ); // Column, Center, Scaffold, MaterialApp
    }
  }

  main() => runApp(Counter());
```

Stateful Widget



- Giải thích ví dụ
 - **Counter** là widget (thứ được dùng để chèn vào widget tree) trong khi **_CounterState** là trạng thái.
 - **_CounterState** là trạng thái có thể thay đổi (mutable state) của widget **Counter**.
 - Trạng thái tồn tại để xây dựng lại nhưng phương thức **build()** của nó thì không.
 - Khi Flutter làm tươi (refresh) giao diện người dùng, phương thức **build(...)** của **State<Counter>** sẽ được gọi.
 - Các thuộc tính của **State<Counter>**, chẳng hạn như **_counter** không bị mất giá trị. Chỉ những gì bên trong phương thức **build()** mới bị refresh.
 - Phương thức **setState(...)** được dùng để chạy lại phương thức **build()**
 - Các lớp con của **State<Counter>** có quyền truy cập vào phương thức **setState(...)** để refresh lại widget.

Widget Row

- **Widget Row** đặt một hoặc nhiều phần tử con vào trục ngang với các thiết lập về cách sắp đặt.
 - Các widget trong row (hàng) có thể được đặt theo nhiều cách khác nhau tùy thuộc vào giá trị của **mainAxisAlignment**. Giá trị mặc định là **MainAxisAlignment.start**.
 - Các giá trị của **mainAxisAlignment**
 - **MainAxisAlignment.center**



Widget Row



- **Widget Row** đặt một hoặc nhiều phần tử con vào trục ngang với các thiết lập về cách sắp đặt.

- Các giá trị của **mainAxisAlignment**

- **MainAxisAlignment.start**



- **MainAxisAlignment.end**



- **MainAxisAlignment.spaceAround**



- **MainAxisAlignment.spaceBetween**



Widget Column

- **Widget Column** đặt một hoặc nhiều phần tử con theo trục dọc với các thiết lập về cách sắp đặt.
 - Các widget trong column (cột) có thể được đặt theo nhiều cách khác nhau tùy thuộc vào giá trị của **mainAxisAlignment**.



start



center



end



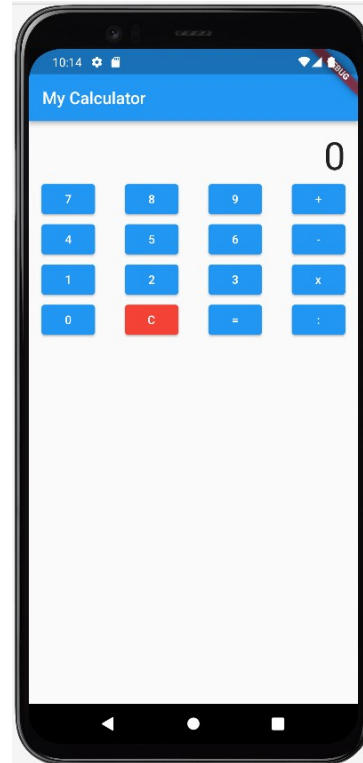
between



around

Bài tập

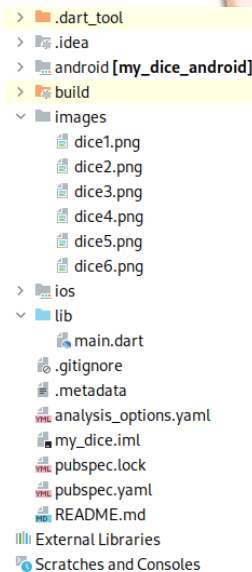
- Xây dựng App máy tính như hình vẽ sau



Chèn ảnh vào App

- Tạo một thư mục cùng cấp với thư mục **lib** trong project dùng để chứa ảnh. Thường đặt mặc định tên là **images**.
- Copy các ảnh muốn dùng cho app vào thư mục images này.
 - Bên trong thư mục có thể tiếp tục phân cấp (nếu cần)
- Bên trong file **pubspec.yaml**, thêm các dòng sau vào dưới mục **flutter** (lùi vào 1 tab) để báo cho trình biên dịch biết nơi chứa ảnh của app:

```
flutter:  
  uses-material-design: true  
  assets:  
    - images/
```



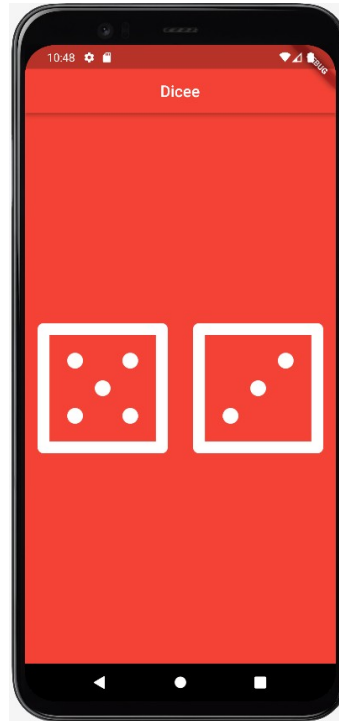
Chèn ảnh vào App

- Sử dụng Widget sau để chèn ảnh vào app
Image.asset(
 'images/my_image.png',
),
- Ví dụ:

```
Expanded(  
  child: MaterialButton(  
    onPressed: () => randomizeSynced(),  
    child: Image.asset(  
      'images/my_image.png',  
    ), // Image.asset  
  ), // MaterialButton  
) // Expanded
```

Bài tập

- Xây dựng App đồ xúc sắc như sau



Q & A

